CHAPTER 8: YEAR-1 CORE, QUALITY & BASIC SECURITY

Does it *work* — and can it be *trusted*?

Quality is not luck and security is not an add-on. You **check** quality with reviews, tests and tools, and you protect users with a few habits any beginner can apply today — including the law: Thailand's **PDPA**.

TODAY: ONE CHAPTER, DONE PROPERLY

Today: Ch8, then exam practice

Ch8 Quality & Security
3 concept checks
→
Critique #8
ask questions
→
Exam practice
4 MCQ on Ch8
→
Next class
Ch9 + Ch10

GOAL

Understand the concepts

Ch8 is on the final exam (Ch1 to Ch10, English MCQ with 4 options + short answer).

CONCEPT CHECKS

Answer together

After each topic: a short situation, the whole room answers at once. Not graded.

BRING

One A4 sheet

For Critique #8. Handwritten, handed in during class.

Ch9 and Ch10 are next class: **Mon 26 Oct, starting 13:00**.

THE GOLDEN THREAD: ONE STEP FURTHER

Quality & security are how the prototype earns trust



PART 1

Quality

What “quality” means, and how you actually check it — reviews, tests, linters.

PART 2

Basic security

The big three for beginners, plus a real vulnerability you can find today.

PART 3

Privacy & PDPA

Personal data is a legal duty — consent, minimise, protect.

Understand-only this term — these are habits you apply to the prototype and are assessed on the exam, not a separately graded security audit.

PART 1: WHAT QUALITY MEANS

Quality is several attributes — and each is checkable

RELIABILITY

It keeps working

Same input → same correct result, even on a bad day. Checked by tests.

USABILITY

People can use it

Clear, forgiving, learnable. Checked by watching a real user try it.

MAINTAINABILITY

Easy to change

Readable code future-you can edit. Checked by reviews & linters.

SECURITY

It protects data

Only the right people see the right data. Checked by thinking like an attacker.

PERFORMANCE

Fast enough

Responds within the NFR budget. Checked by measuring, not guessing.

THE POINT

You *check*, not hope

Every attribute traces back to an NFR in your SRS — and to a way to verify it.

> IN PLAIN TERMS
> Quality isn't one thing — it's a **checklist**, like judging a restaurant: taste, speed, cleanliness, price. Each is real, and each is checked a different way.

PART 1B: A STANDARD VOCABULARY (ISO/IEC 25010:2023)

Quality has an international model: nine characteristics

#1
Functional suitability
Does what's needed, correctly.

#2
Performance efficiency
Fast enough, within resources.

#3
Compatibility
Works alongside other systems.

#4
Interaction capability
People can learn and use it.

RENAMED: was Usability

#5
Reliability
Keeps working over time.

#6
Security
Protects data and access.

#7
Maintainability
Easy to change safely.

#8
Flexibility
Adapts to new environments and needs.

RENAMED: was Portability

#9
Safety
Avoids harm to people, property and the environment.

NEW in 2023

> WHY IT CHANGED
>
> The 2023 edition (replacing 2011) renamed Usability and Portability and added **Safety**, because software now runs cars, medical devices and AI systems. Each characteristic in your SRS becomes an NFR you can check.

Source: **ISO/IEC 25010:2023** Product quality model, Edition 2 (iso.org/standard/78176.html). Sommerville 9e Ch24 describes the earlier 2011 model.

PART 1: CHEAP, POWERFUL QUALITY TOOLS

Code review & the linter catch problems early

HUMAN

Code review

A teammate **reads your code** before it reaches `main` — this is what a Pull Request is for (Ch6). Catches logic bugs, unclear names, missing cases, and spreads knowledge across the team.

AUTOMATIC

Linter / static analysis

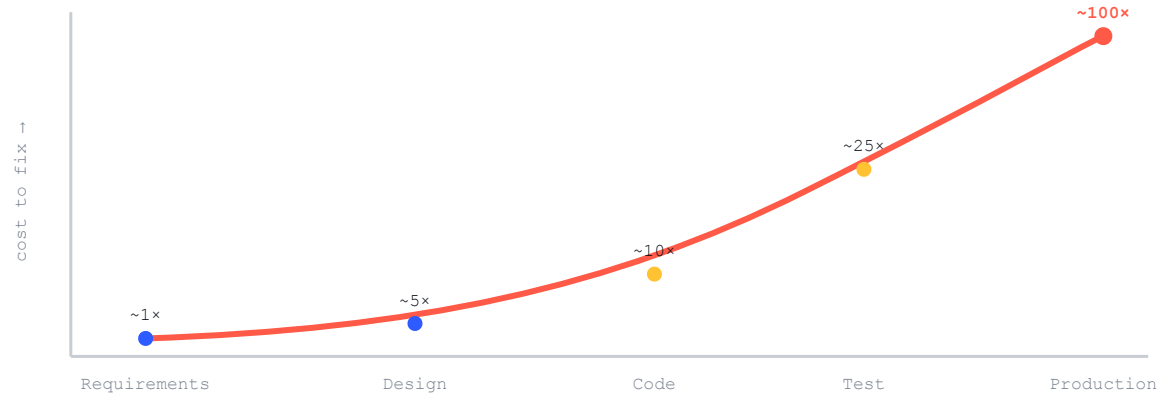
A tool that **reads the code without running it** and flags style, unused variables, likely bugs, risky patterns. Fast, consistent, runs on every commit.

IN PLAIN TERMS

A linter is **spell-check for code**; a code review is a **classmate proof-reading your essay**. Neither is optional on a real team — and both make AI-generated code far safer to keep.

PART 1C: WHY CHECK EARLY (BOEHM)

The later you catch a defect, the more it costs



Illustrative scale — a bug caught in review/test is cheap; the same bug in production is not

After **Boehm (1981) Software Engineering Economics**. This is why reviews (here) and unit tests (Ch7) pay for themselves — they move the catch leftward.

> IN PLAIN TERMS
>
> A typo you catch *while writing* costs nothing; the same typo found *after the book is printed* means a reprint. Bugs are identical — catching them early, in a review or a test, is why both are worth the time.

CONCEPT CHECK 1: QUALITY ATTRIBUTES

Which attribute is it, and how do you check it?

SITUATION	ATTRIBUTE	CHECK IT BY
The app takes more than 5 seconds to respond when many people use it.	**Performance**	Measuring, not guessing
A new user cannot find the Save button.	**Usability**	Watching a real user try it
Changing one feature means editing 10 files.	**Maintainability**	Code review + linter
The calculation is sometimes wrong.	**Reliability**	Tests
User A can see User B's data.	**Security**	Thinking like an attacker (access control)

FOLLOW-UP

If that last bug reaches production, is it more expensive than catching it in review? **Yes, by an order of magnitude or more (Boehm): the later you find it, the more it costs.**

PART 2: SECURITY YOU CAN APPLY TODAY

Three habits that stop most beginner mistakes

#1

Validate input

Never trust data from the user or the browser. Check type, length and range on the **server**, before you use or store it.

#2

Access control

A logged-in user may see **only their own data**. Check “who are you, and is this yours?” on every request.

#3

No secrets in the repo

Passwords, API keys, tokens never go in the code or Git history. Use `.env` + `.gitignore` (Ch6).

IN PLAIN TERMS

Picture a shop: check what comes through the door (**validate input**), only staff can open the till (**access control**), and never tape the keys to the door (**no secrets in the repo**).

PART 2: TRUSTING THE CLIENT = THE CLASSIC HOLE

If the user can change the id, they can read your data

Wrong: The server trusts an id from the URL

```
# anyone logged in can swap the number
GET /api/notes?user_id=1042

def get_notes(user_id):
  return db.notes.where(owner=user_id)
  # ...whose id is this? nobody checked.
```

Right: Use the identity from the session

```
# id comes from WHO is logged in
GET /api/notes

def get_notes(session):
  me = session.user   # trusted
  return db.notes.where(owner=me.id)
```

Change `1042` to `1043` → you're reading someone else's notes. This is **IDOR** — broken access control.

THE RULE

Never decide “whose data” from something the user can edit. Derive it from the authenticated session.

CONCEPT CHECK 2: THE BIG THREE

Which habit was missing? Show 1, 2 or 3

1. Validate input	2. Access control	3. No secrets in repo
on the server	who are you, is it yours?	.env + .gitignore

SITUATION	ANSWER
The age field accepts `-5` and the system saves it.	**1. Validate input**
Changing `order_id` in the URL shows someone else's order.	**2. Access control (IDOR)**
`config.js` in a Public repo contains an API key.	**3. No secrets in the repo**
Password length is checked only in the browser with JavaScript; the server accepts anything.	**1. Validate input: check on the server, because users can bypass the browser**

PART 3: A LEGAL DUTY, NOT JUST GOOD MANNERS

Personal data comes with responsibilities

CONSENT

Ask first

Collect personal data — schedules, notes, messages, faces — only with the person's informed consent, for a stated purpose.

MINIMISE

Collect the least

If the feature doesn't need it, don't store it. Less data = less risk and less to protect.

PROTECT

Guard & honour rights

Secure it, and let people see, correct or delete their data. This is **PDPA** — Thai law since 2022.

FOR YOUR PROJECT

Use **fake / seed data** in the prototype. Real personal data belongs nowhere near a student repo — and never in Git history.

CONCEPT CHECK 3: PDPA

Consent, minimise, or protect?

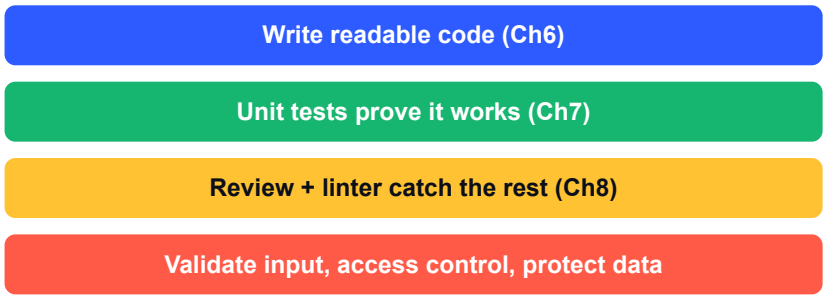
SITUATION	PRINCIPLE
A note-taking app asks to read your whole contact list, but no feature uses it.	**Minimise**
An app stores photos of users' faces without saying what they are for.	**Consent (and state the purpose)**
A user asks to delete their account, but the system has no way to delete their data.	**Protect: the owner's rights to see, correct and delete**

REMEMBER

PDPA (B.E. 2562, fully in force 2022): **consent** before collecting, **minimise** what you keep, **protect** it and honour the owner's rights.

HOW THE CHECKS STACK UP

Quality & security are layers, not a final step



Each layer catches what the one above let through — trust is built up, not bolted on

DO THIS WITH YOUR TEAM

Make quality & safety part of the build

PROJECT STEP

Add a linter on day one

Let it run on every commit — cheap quality your whole team gets for free.

Check access on every read

Whose data is this? Derive the owner from the session, not the request.

AI-FIRST — VERIFY!

Ask AI to review your code

Great at spotting smells — but confirm each flag; it also invents some.

Never paste real secrets/data

Don't feed API keys or real personal data into a prompt — treat it as public.

REFERENCES: WHERE THIS COMES FROM

Textbook & source references

1 **Sommerville, I. (2011) Software Engineering, 9e**
Ch24 Quality management; Ch14 Security engineering (§14.1 risk, §14.2 design for security); §11.4; §13.4.

2 **ISO/IEC 25010:2023**
Product quality model, Edition 2 (replaces 2011): nine characteristics, Safety is new.

3 **OWASP Top 10 (2021)**
The most critical web-app risks — Broken Access Control is #1 (the IDOR demo).

4 **Boehm, B. (1981) Software Engineering Economics**
Cost of fixing a defect rises sharply across the life cycle.

5 **PDPA B.E. 2562 (2019)**
Thailand's Personal Data Protection Act — in force 2022.

Read Ch24 + skim the OWASP Top 10 before the exam.

CHAPTER 8: IN FOUR LINES

Check quality, don't hope for it — and protect the user

QUALITY

Attributes you can verify

Reliability, usability, maintainability, security, performance — each traces to an NFR and a check.

TOOLS

Review + linter

A human reads it; a tool scans it. Cheap, early, on every commit.

SECURITY

The big three

Validate input, enforce access control, keep secrets out of the repo.

PRIVACY

PDPA is law

Consent, minimise, protect — and keep real personal data out of the prototype.

Reading: Sommerville (quality & security), OWASP Top 10 (skim), PDPA overview. Next: Ch9 — Project Management & Process.

AI CRITIQUE #8: ASK, DON'T ANSWER (7 MIN, HANDWRITTEN A4)

Ask questions about a project like yours

QUESTION FOCUS

Imagine a student project from this course, **CareFlow OPD** (outpatient queue booking), has been live at a hospital for one month. A patient posts on Facebook: "Change the appointment number in the link and you can see other patients' names, phone numbers and symptoms. I even cancelled someone else's appointment." The team says every FR was tested and all tests passed. The repo is public, and `config.js` contains the database password.

สมมติว่าโปรเจกต์นักศึกษา CareFlow OPD (ระบบจองคิวผู้ป่วยนอก) เปิดใช้จริงในโรงพยาบาลมา 1 เดือน มีผู้ป่วยโพสต์ใน Facebook ว่า "ลองเปลี่ยนเลขนัดในลิงก์ดู จะเห็นชื่อ เบอร์โทร และอาการของผู้ป่วยคนอื่น ผมยังยกเลิกนัดของคนอื่นได้ด้วย" ทีมบอกว่าทดสอบทุก FR แล้วและเทสต์ผ่านหมด repo เป็น Public และไฟล์ config.js มีรหัสผ่านฐานข้อมูลอยู่

1. **Write at least 3 questions**
   About the event. Do **not** answer them. End each with (closed) = short answer, or (open) = needs explaining.

2. **Circle the most important one**
   Write 1 to 2 lines: why this question matters most.

3. **Link it to one Ch8 concept**
   Name one concept that helps answer your circled question (e.g. access control, PDPA, cost of defects) and say in one line how.

7:00

Top of the A4: name, student ID, **Critique #8**. Alone, handwritten, phones and laptops face down until the sheet is collected. There are no wrong questions.

CASE: CAREFLOW OPD, A WHAT-IF ON A STUDENT PROJECT

Four Ch8 failures in one small system

ACCESS CONTROL (IDOR)

Change the number, see another patient

The server trusted the appointment id in the link. Fix: take the patient from the login session and check the appointment is theirs, for viewing **and** for cancelling.

PDPA: MINIMISE, PROTECT

Health data is sensitive

Symptoms are health data, a sensitive category under PDPA that needs explicit consent. Does the booking page need symptoms at all? Show the least, protect the rest.

QUALITY: CHECK, DON'T HOPE

"All tests passed"

The tests only tried honest users. Add the attacker's case: patient A asks for patient B's appointment, and the system must refuse.

NO SECRETS IN THE REPO

A password in a public repo

Anyone could read `config.js`. Revoke the password and keep secrets in `.env` listed in `.gitignore`. Deleting the line is not enough: Git history keeps it.

THE LESSON

You don't need a giant company to break every rule in Ch8. A student-sized system with real patients needs the **big three** and **PDPA** from day one.

What-if scenario built on a student project idea (Must FRs: book, view, cancel appointments). It does not describe any team's actual prototype. PDPA B.E. 2562, Section 26: health data is sensitive personal data.

CRITIQUE #8: LINK YOUR QUESTION TO THE CHAPTER

Every good question leads back to a Ch8 concept

IF YOU ASKED	CH8 CONCEPT	WHAT REALLY HAPPENED
Why could changing the number show someone else's data?	**Access control (IDOR)**	The server trusted the id in the link
Does the booking page need symptoms at all? Who agreed to share them?	**PDPA: minimise, consent for sensitive data**	Health data visible to anyone with a link
How did every test pass?	**Quality: check, don't hope**	The tests only tried honest users
Why was the password in the repo?	**No secrets in the repo**	`config.js` in a public repo
What does fixing it after launch cost, compared with at design time?	**Cost of defects**	A code fix, plus a data review and telling patients

THE WHOLE CHAPTER IN ONE CASE

The big three, PDPA, and testing the dishonest case. The best question is often the one the slide cannot answer yet, like **"Who in the team should check the access rules before launch?"**

EXAM PRACTICE: CH8 MCQ 1 TO 4 (ALONE, ON YOUR OWN PAPER, NOT GRADED)

Practice like the real exam

1. Which of the following is not one of the "big three" security habits taught in this course?

A. Validate input on the server.

B. Enforce access control on every request.

C. Keep secrets out of the repository.

D. Use a dark theme to hide sensitive fields.

2. A user changes `user_id=1042` to `user_id=1043` in the URL and sees another person's notes. What is the best fix?

A. Hide the id from the URL bar.

B. Take the owner's identity from the authenticated session.

C. Check that the id is a number.

D. Add a linter to the project.

3. Which is not a characteristic in the ISO/IEC 25010:2023 product-quality model?

A. Interaction capability.

B. Safety.

C. Profitability.

D. Maintainability.

4. According to Boehm's cost-of-defects idea, where is a defect usually cheapest to fix?

A. Requirements.

B. Testing.

C. Production.

D. Maintenance.

About 4 minutes. Answers are revealed one by one afterwards.

THAT'S CH8. SEE YOU NEXT CLASS

Next: Ch9 and Ch10

NEXT CLASS

Mon 26 Oct, 13:00 to 14:50

Note the later start: **13:00**, not 12:00. Ch9 Project Management and Ch10 Professional Ethics.

BRING

Two A4 sheets

Critique #9 and Critique #10, handwritten in class.

HAND IN TODAY

Critique #8 sheet

Your A4 has been collected. Name, student ID and "Critique #8" on top.

BEFORE NEXT CLASS

Review Ch8

The Ch8 and Ch9 self-study handbook will be posted in Classroom, with self-checks and practice MCQs.